Four Languages, One Structure

The Computational Grounding of ⊥

ZP Companion | Initial: April 2026 | Current: 1.26, November 2026

This companion explains the ideas in plain language. It is not the formal document. Its claims about the Lean development restate results proved in the technical document ZP-K Computational Grounding or in the corpus it cites; readings and commitments are labelled as such, and the remaining background is standard computability theory and logic, named where it is used. Consult that document for the authoritative mathematics.

What Is ZP-K Doing?

ZP-J proved that the Quine atom (set-theoretic self-reference) and the bottom element (order-theoretic minimum) are the same object. ZP-K adds a fourth language: computability theory. It adds a fourth description of ⊥, this time in terms of Turing machines and Kleene's second recursion theorem. THREE of the four descriptions are proved equivalent: T-COMP shows that, for an element of a ZPSemilattice carrying KleeneStructure, being a Quine atom is equivalent to being the semilattice's least element ⊥ together with the join identity, and da2_bottom_characterization shows those two equivalent. ZP-K carries c_0's computational face as a KleeneStructure requirement (botCode_is_quine); reading c_0 as the Kleene quine is the KleeneStructure commitment, not a theorem.

On the framework's reading, ⊥ is not a description of a Turing machine waiting for an outside party to run it: ⊥ is read as an instance of a Turing machine (its ground state, serving as its own program), so if it runs at all, it runs itself (Path 1, the framework's requirement, not a theorem). That rules out an outside executor, not an idle ⊥: ⊥ is the unique self-containing state of MachinePhase (da1_closed_concrete) even in a dynamics where nothing ever leaves c_0 (tsnap_holds_but_nothing_moves). That ⊥ does run is the occurrence commitment, which DA-1 uses and does not supply.

What Is a Kleene Fixed Point?

Kleene's second recursion theorem is a foundational result in computability theory. Informally, it says: for any computable way of transforming programs, there exists a program that is a fixed point of that transformation — a program whose behavior is the same before and after the transformation is applied.

A Kleene fixed point of self-application is called a computational Quine: a program c such that running c produces the same output as running c on its own source code. The definition says only that: it fixes what c does, and it says nothing about minimality — there is no claim here that no shorter program generates c, which would need Kolmogorov complexity and plays no formal part in this document.

This is not a running computation that might loop forever. Kleene's theorem is an existence proof — it guarantees a fixed point exists before any execution takes place, by a direct construction, not by iterating toward a limit. The question "will it halt?" does not arise: the fixed point is identified by the theorem itself, not discovered by running a potentially divergent process.

Real-world analogy — A self-printing program

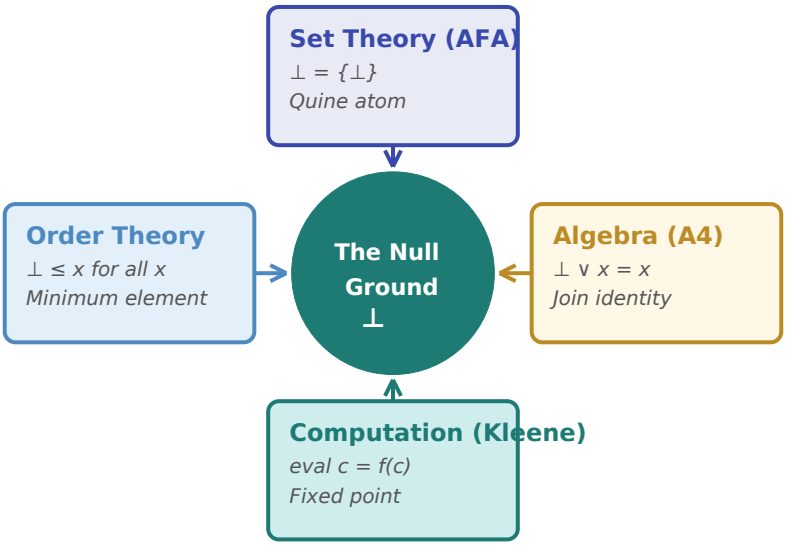
A Quine program in computer science is a program that, when run, outputs its own source code. It needs no external file to read — the source is baked in. Kleene's theorem guarantees that every partially computable map f sending each program to a partial function has a fixed point, a program c whose behaviour eval c is the function f sends it to, eval c = f c (kleene_fixed_point_exists); a self-printing program of this kind is a fixed point of one particular such map (selfref_universal_exists, below). On the framework's reading, the computational Quine is the expression of $\bot = \{\bot\}$ in the language of programs: c is its own program, just as $\bot$ is its own member. ZP-K carries that reading as a KleeneStructure requirement (botCode_is_quine), not a theorem.

The periodicity condition that defines a computational Quine is a weak test: a program that ignores its input and always prints the same number passes it. A sharper test is not passed that way. A self-printing program returns its own code number on one channel; a universal one computes what any other program computes, given that program's number, on the remaining channels. Programs that are both exist, and there are infinitely many of them (selfref_universal_exists, selfref_universal_infinite); no constant program is universal. More generally, every partially computable map of that kind has infinitely many fixed points (fixed_points_infinite), and every computable function has infinitely many programs (padding).

Two self-printing programs that behave identically are the same program (selfprints_behaviour_injective): the self-printing channel reads the program's own number back out of its behaviour. The framework's reading: computational self-reference has a uniqueness, at the level of behaviour. Both halves stand: there are infinitely many self-printing programs, and no behaviour belongs to two of them. For the weaker periodicity test, "computational Quines are not unique" stays true. None of these results says that a program is run; they say that such programs exist.

T-COMP: The Three-Way Equivalence, and What Computation Adds

ZP-K's central theorem, T-COMP (Computational Grounding), with da2_bottom_characterization, proves that THREE of the four descriptions below are equivalent - self-containment, order, and algebra all identify the same object. The fourth, the computational one, is not a clause of the theorem: it is supplied as an assumption when the structure is built. The framework holds to that distinction:



The null ground element ⊥ at the center, described in four formal languages. The arrows among the first three are proved equivalences. The computational description is joined to them by assumption, not by proof.

Language	Property	Intuition
Set theory (AFA)	⊥ = {⊥} — Quine atom	⊥ is its own sole member; no external interpreter exists
Order theory (ZP-A)	⊥ ≤ x for all x — minimum	⊥ is below everything; the universal starting point
Algebra (ZP-A A4)	⊥ ∨ x = x for all x — join identity	⊥ contributes nothing to any join; the additive zero
Computation (Kleene)	eval botCode = selfApply botCode — fixed point	a fixed point of self-application; reading it as "⊥ is its own program" is the framework's commitment, not a consequence

The framework READS the four descriptions as projections of a single structural identity rather than as independent convergences - that the Kleene fixed-point property and the AFA Quine atom property are one property in two vocabularies. That reading is the motivation for building a KleeneStructure, the formal bridge connecting the set-theoretic (AFAStructure) and computational (Kleene fixed-point) descriptions. It is not something ZP-K derives: the bridge is built because the reading is held, not the other way round.

> Remember: T-COMP is not saying that set theory and computability theory are the same. It is saying that the specific structural role played by ⊥ — self-containment, minimality, join-identity — is expressible in several languages, and that THREE of those expressions are provably equivalent; the computational one is joined by assumption. The equivalence is local to this structural role, not a global identification of the frameworks.

DA-1: what Lean witnesses

DA-1 (Instantiation as Execution) had three informal argument paths in ZP-E:

Path 1 (Structural — AFA): Nothing external to ⊥ can execute ⊥, so if ⊥ executes at all, it executes itself. That rules out an outside executor; it does not rule out an inert ⊥. ZP-J proved axiom-free that ⊥ is the unique self-containing element (the structural fixed point); that this is the literal ⊥ = {⊥} holds in the ZF+AFA setting. What Lean witnesses via ZP-K: Path 1's Lean counterpart is da1_closed_concrete, which proves IsQuineAtom (⊥ : MachinePhase) through the KleeneStructure instance machinePhaseKleene, whose AFAStructure part is machinePhaseAFA. There selfMem is defined as x = ⊥, so the content is the uniqueness rather than the membership, and the same statement has a proof from machinePhaseAFA alone (the example after da1_closed_concrete in Kleene.lean).

Path 2 (Informational — L-INF): The surprisal of ⊥ is unbounded — no finite interpreter can hold it as a stored description. Remains outside Lean scope: the step from "unbounded surprisal" to "executing" is a bridge principle of its own, which the framework does not adopt, and it is not the occurrence commitment.

Path 3 (Computational — Kleene): No shorter program generates ⊥. That rules out a shorter outside generator, not an inert string, so Path 3 does not derive executing from incompressibility. The reading that ⊥ is its own program — a Kleene fixed point of self-application — is not a Lean result. ZP-K supplies botCode_is_quine as a requirement of the KleeneStructure class, assumed when the structure is built rather than derived; and the condition it asks for is a periodicity condition that even a constant code meets. The "no shorter program" reading needs Kolmogorov complexity, which is uncomputable and plays no formal part here. None of the three paths derives what DA-1 needs: that the configuration reaching P_0 is running is the occurrence commitment, which DA-1 uses and does not supply (ZP-E).

da1_closed_concrete (Kleene.lean)

IsQuineAtom(⊥ : MachinePhase) — proved in Lean 4. The initial machine state c_0 is self-containing. The Lean proves nothing about execution: that the configuration reaching P_0 is running, not an inert description, is the framework's occurrence commitment (ZP-E). DA-1 Path 1 (structural): its Lean counterpart is proved, and Path 1 rules out an external executor, not an inert ⊥. Path 3: its Lean counterpart is a KleeneStructure assumption, not a derivation. Path 2: outside Lean scope (an unadopted bridge principle).

What does it mean, on the framework's reading, that ⊥ IS an instance of a Turing machine in its ground state? In ZP-C, the model distinguishes c_0 (the initial configuration, before any instruction executes) from c_1 (after the first instruction fetch). DP-2 (ZP-E) proved that these are distinct machine states even when both produce the same output value. On the framework's reading, ⊥ is not a description waiting for someone outside to press "run": if it runs at all, it runs itself (Path 1). That it does run is the occurrence commitment, which DA-1 uses and does not supply; given it, the machine has left c_0 for c_1, by the definition of the first running configuration (D7). ZP-K carries c_0's computational face as a KleeneStructure requirement (botCode_is_quine); reading c_0 as the Kleene quine is the KleeneStructure commitment, not a theorem. What T-COMP, with da2_bottom_characterization, proves is narrower: three of the four descriptions of ⊥ are equivalent to one another. It says nothing about external agents, and nothing about execution.

Real-world analogy — The light that sees itself

Consider a camera that, instead of photographing external scenes, photographs only its own sensor. The image it produces is the state of the sensor; the sensor's state is the image. There is no external scene being captured — the camera IS the scene. On the framework's reading, ⊥ as a Kleene fixed point has exactly this structure: the program that runs is the program that describes what runs, so if it runs, nothing outside it is needed to run it. The analogy stops there: the same camera, wired the same way, can also sit switched off. That ⊥ does run is the framework's occurrence commitment, not something the self-reference supplies.

A Note on Proof Purity

The statements of Kleene's theorem and of Rogers' fixed-point theorem, as Mathlib states them, already carry Classical.choice (measured 2026-10-04), so every proof of them as stated carries it too: the STATEMENT-CARRIED class of ZeroParadox/Category/ChoiceCannotBe.md. That footprint is inherited from Mathlib rather than a Zero Paradox commitment. Kleene's theorem restated with the transformation given as a code has a statement and a proof free of choice (kleene_cf, ZeroParadox/Computability/KleeneChoiceFree.lean); Rogers' form has no such restatement in this corpus. ZP-K Section IV tabulates the measured axiom footprints. What is free of Classical.choice is the choice-free CORE, and that is narrower than any layer name: ZeroParadox/AxiomProfile.lean §I is where the core is listed and each footprint measured — T-SNAP, the lattice, the Quine atom. Results elsewhere in the same layers do carry the dependency: decoration_unique in ZeroParadox/Settheory/APG.lean, a ZP-J theorem that two decorations of a finite accessible pointed graph into a DecorationUniverse are equal, reports the full triple. AFA itself is an axiom, "Every graph has a unique decoration" (Aczel 1988, p. 6), not a theorem. Check the artifact for a given result rather than reading a layer name as a footprint.

Using a result from the standard, classically built libraries puts a proof in the classical setting those libraries are built in. Whether a given footprint is needed is a separate question, answered declaration by declaration, and ZP-K Section IV records the measurements rather than a rule.

Remember: in ZP-K, DA-1 Path 1's Lean counterpart is da1_closed_concrete, which proves IsQuineAtom(⊥ : MachinePhase) and nothing computational; Path 3's Lean counterpart is the machinePhaseKleene instance's botCode_is_quine field, a KleeneStructure requirement that constant codes also meet, not a witness of execution and not a second independent proof. The two are carried together by da1_paths_unified as a conjunction; that they name one structural fact is the framework's reading. ZP-K does not claim to resolve the description-execution gap by philosophical argument.

Self-Reference: Fixed Point vs. Oscillation

Not all self-reference is the same. The liar paradox — "this sentence is false" — produces a statement x with the property x = NOT x. In Boolean logic this has no fixed point: the sequence true, false, true, false, ... oscillates without resolving. This is the liar structure.

Gödel’s incompleteness proof (1931) uses a different kind of self-reference. Using the diagonal lemma — a fixed-point theorem for formal languages — he constructed a sentence G satisfying G ↔ ¬Prov(G): "this sentence is not provable in PA." The key move: he changed the predicate from "true" to "provable." Provability is asymmetric in a way truth is not, so G does not oscillate. In a consistent system, G is true in the standard model of arithmetic but unprovable in PA — a fixed point, not an oscillation.

ZP-K's construction achieves the same structural form in the setting of ⊥. ⊥ = {⊥} (the AFA Quine atom; da1_closed_concrete gives this for MachinePhase, where selfMem is defined as x = ⊥, so the content is the uniqueness rather than the membership) is self-containing, not self-negating: x = f(x) where f is set-membership, not x = NOT x. The Kleene fixed point gives the same structure computationally: a code whose behavior is determined entirely by itself. Both witnesses resolve at a fixed point. Neither oscillates.

ZP-E's t_snap_irreversible goes further: it formally proves that the liar-type trajectory is structurally impossible in this framework, not merely absent. Once c_0 transitions to c_1, the semilattice axioms guarantee no path returns. The sequence $c_0 \to c_1 \to c_0 \to \ldots$ cannot occur — the algebra forbids it.

> This is a structural observation, not a claim that ZP proves Gödel's theorems. Gödel applied the diagonal lemma in formal arithmetic; ZP-K's construction achieves the same structural form — x = f(x) rather than x = ¬x — for the bottom element ⊥. Both constructions use x = f(x) rather than x = ¬x, and both yield a stable fixed point rather than an oscillating sequence.

Replication: Presence, Address, and Selection

A description can be used in two ways. Cotler, Hongler and Hudcová (2025) describe von Neumann's self-replicating automaton as using its description twice: once interpreted, to build, and once transcribed, to hand to the copy. The framework's reading: in ZP-E's terms the transcribed use is a description sitting as data (Sense A) and the interpreted use is a description being run (Sense B). Section VI of the formal document follows one arc: every instruction is present on a tape, self-printing interpreters are told apart only by an address, such a tape holds no full copy of itself, and building an address differs from choosing one. The paragraphs below take it in that order.

Every instruction is present. A tape of bits on which every finite pattern appears is called disjunctive. On such a tape the code of every program appears, infinitely often (code_occurs_of_disjunctive). That is presence, not execution. It needs no randomness: a simple counting tape in the style of Champernowne is disjunctive and computable. The framework holds, as a commitment, that the occupant of its ⊥ role, ⊥ of a ZPSemilattice, read as a tape, is maximally complex; that commitment is about that occupant, not about the all-false tape below. Standard theory, not proved in this corpus: a sequence whose every prefix is incompressible up to a constant in the prefix-free sense of Kolmogorov complexity is exactly a Martin-Löf random one (the Levin–Schnorr theorem; Franklin and Porter, arXiv:2004.02851, Theorem 2.5; proved in Lean outside Mathlib in a development that the ZPSurprisal entry of LEAN_CUSTOM_REGISTRY.md names), and such a sequence is disjunctive. The prefix-free sense matters: with plain complexity every infinite sequence has infinitely many prefixes of length n compressible to at most $n - \log n$ bits (Franklin and Porter, arXiv:2004.02851, § 2.2, p. 15, crediting Martin-Löf; Martin-Löf 1971, Theorem 1, proves the form conditional on the length n).

Two self-printing interpreters differ only on channel 0. Two self-printing universal programs give the same answers on every channel where they compute other programs, and differ only on the channel where each prints its own number (selfPrints_universal_address). The framework's reading, not a theorem: such programs are replicas, sharing everything but the address, and each new one adds only its address.

No full copy of itself. A disjunctive tape never equals itself shifted along by any positive amount (disjunctive_not_periodic). The reading: copies sit side by side, told apart by an address, rather than one tape containing itself whole.

Building an address versus choosing one. The recursion theorem produces a self-printing universal program (selfref_universal_exists). What Lean has about choosing is about a two-way choice, between two charts labelled by Bool (ZeroParadox/Valuation/PoleChartSelection.lean), and it has two halves. For an inhabited test that is decidable in Lean's sense, which is not the same as having an algorithm, an if picks a witness (select_of_decidable). A picker that works uniformly for every inhabited test on Bool is, by definition, a fragment of the axiom of choice (uniformChartSelection_iff_choiceFragment). The framework's reading, by analogy and not a theorem about programs: building a program is a computation, and choosing a program by what it does is where choice would do its work. In Lean the footprint does not draw that line: the existence theorem's statement carries Classical.choice (STATEMENT-CARRIED in the sense of ZeroParadox/Category/ChoiceCannotBe.md), while a restated statement, with the Gödel number and the channels spelled without Mathlib's numbering of programs, is free of choice and has a proof free of choice (selfref_universal_exists_enc, ZeroParadox/Computability/UniversalChoiceFree.lean; the record is ZeroParadox/Computability/Kleene.md § VIII).

The zero tape. The all-false tape is ⊥ of the bit-by-bit tape order, carries no program's code except the empty one, and is what any tape becomes when compared with itself bit by bit. The framework reads it two ways at once: the reference every comparison runs through, and every tape's difference from itself. The commitment above calls the occupant of the framework's ⊥ role, ⊥ of a ZPSemilattice, maximally complex; this all-false tape is ⊥ of a different structure, tapes under the bit-by-bit order, and the commitment is not about it.

Replication beyond self-reference. Cotler, Hongler and Hudcová build an automaton that is universal locally and still cannot copy itself, because any of its machines that meets a cell another machine has marked halts at once, so no information passes between them. As they put it, a quine "produces a description of the copying mechanism without replicating the mechanism itself". In one example, occurrence is not visible from outside either: the carry example in Occurrence.lean (carry_steps_onward_forever_yet_shows_nothing) steps onward forever while one observable shows the same thing at every step. The reading: watching that observable cannot tell this example from a machine that never moves. Lean shows this for that one example and one observable, not for every way of looking.

Remember: presence on a tape, self-printing, and universality are all facts about descriptions and what programs compute. Two readings are both kept: a complete description of a run is the run (history view), and that a configuration is running is a further fact (state view), the occurrence commitment, which DA-1 uses and does not supply.

Key Result — ZP-K

T-COMP: IsQuineAtom(q) ↔ q = ⊥ ∧ (∀ x, q ∨ x = x). THREE characterisations are proved equivalent - self-containment (AFA), order, and algebra (the last two by da2_bottom_characterization). The computational face is NOT a fourth clause of this theorem: it enters as an assumption, the KleeneStructure field botCode_is_quine. da1_closed_concrete : IsQuineAtom(⊥ : MachinePhase) proves the structural half only - it mentions no code and no execution. DA-1 Path 1 (structural): its Lean counterpart is proved, and Path 1 rules out an external executor, not an inert ⊥. Path 3 (computational) does not derive executing from incompressibility; its Lean counterpart is a class field, assumed rather than derived.